
1. SQL Joins

Joins are used to **combine rows** from two or more tables based on a related column.

Types of Joins:

Join Type	Description
INNER JOIN	Returns only matching rows from both tables.
LEFT JOIN (OUTER JOIN)	Returns all rows from the left table and matching rows from the right table (NULL if no match).
RIGHT JOIN (OUTER JOIN)	Returns all rows from the right table and matching rows from the left table (NULL if no match).
FULL JOIN (OUTER JOIN)	Returns all rows from both tables (NULL where there's no match).
CROSS JOIN	Returns the Cartesian product (each row in Table A is paired with each row in Table B).
SELF JOIN	Joins a table to itself.

Example: INNER JOIN

```
SELECT employees.id, employees.name, departments.dept_name
FROM employees
INNER JOIN departments ON employees.dept_id = departments.id;
```

- ◆ Returns only employees that have a matching department.
-

2. Aggregate Functions

Aggregate functions perform calculations on a set of rows and return a **single value**.

Function	Description
COUNT()	Returns the number of rows.
SUM()	Returns the sum of a numeric column.
AVG()	Returns the average of a numeric column.
MAX()	Returns the highest value.
MIN()	Returns the lowest value.
GROUP_CONCAT()	Concatenates column values into a single string (MySQL only).

Example: COUNT, SUM, AVG

```
SELECT department, COUNT(*) AS total_employees, SUM(salary) AS total_salary, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

- ◆ Groups employees by department and calculates total employees, total salary, and average salary per department.
-

3. ROW_NUMBER()

Assigns a **unique** row number to each record **within a partition**, ordered by a specific column.

Example: Assign a row number to each employee per department

```
SELECT id, name, department,
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num
FROM employees;
```

- ◆ Each department starts numbering from 1, sorted by highest salary first.
-

4. DENSE_RANK()

Similar to `ROW_NUMBER()`, but **does not skip ranks** when there are duplicate values.

Example: Ranking employees based on salary within each department

```
SELECT id, name, department, salary,  
       DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS salary_rank  
FROM employees;
```

♦ If two employees have the same salary, they get the same rank, but the next rank is NOT skipped.

Difference Between ROW_NUMBER() and DENSE_RANK()

Function	Unique Numbers?	Skips Ranks?	Handling Duplicates
ROW_NUMBER()	✓ Yes	✓ Yes	Each row gets a unique number, even if values are the same.
DENSE_RANK()	✓ Yes	✗ No	Ties (duplicate values) get the same rank, next rank continues without skipping.

Example Comparison

ID	Name	Salary	ROW_NUMBER()	DENSE_RANK()
1	Alice	5000	1	1
2	Bob	4500	2	2
3	Charlie	4500	3	2
4	Dave	4000	4	3

♦ `ROW_NUMBER()` assigns unique values. `DENSE_RANK()` keeps ties together and continues ranking without skipping.
